

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A Experiments

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.
2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.
3. Design and implement a secure file-integrity verification system using MD5 and SHA-256. The system should generate hash values for a file, store the reference hashes, and later verify whether the file has been modified."
4. Write a Python program to generate the SHA-256 hash of a given message and demonstrate the avalanche effect by changing one character.
5. Implement MD5 and SHA-256 hashing algorithms using Python's hashlib library. Compare their outputs for the same input
6. Write a Python program to generate the MD5 hash value of a given text message.
Input: "Hello World"
Output: 32-character hexadecimal MD5 digest.
7. Write a Python program to compare MD5 and SHA-256 for the same input message.
Display:
Original message
MD5 digest
SHA-256 digest

Digest length

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT – 2

Title

Implementation of Digital Signature Scheme Using RSA in Python

Aim

To develop a Python program to implement a Digital Signature scheme using RSA and demonstrate message signing and signature verification. The experiment analyzes how digital signatures provide **message integrity, authentication, and non-repudiation**.

Objectives

1. Generate an RSA public-private key pair.
2. Create a digital signature for a given message using the private key.
3. Verify the signature using the corresponding public key.
4. Demonstrate that modification of the message causes signature verification to fail.
5. Understand the role of digital signatures in secure communication.

Software Requirements

- Python 3.x
- **cryptography** library
- VS Code / IDLE / Jupyter Notebook / Google Colab

Theory

A **digital signature** is a cryptographic mechanism used to verify the authenticity and integrity of a digital message or document.

RSA digital signatures use a pair of mathematically related keys:

- **Private Key** – used by the sender to generate the digital signature.
- **Public Key** – distributed to others and used to verify the signature.

The message is first processed using a cryptographic hash function such as **SHA-256**. The resulting hash is then digitally signed using the sender's private key.

The receiver uses the sender's public key to verify the signature.

Security Properties

1. Integrity:

If the message is modified after signing, signature verification fails.

2. Authentication:

Successful verification indicates that the signature was generated using the corresponding private key.

3. Non-repudiation:

The private-key-based signature provides evidence that the signer authorized the message, assuming the private key remains under the signer's control.

Algorithm

1. Start the program.
2. Generate an RSA private key.
3. Extract the corresponding public key.
4. Define the original message.
5. Generate a digital signature using RSA-PSS and SHA-256.
6. Display the original message and signature.
7. Verify the signature using the public key.
8. Modify the original message.
9. Attempt verification again.
10. Display the verification results.
11. Stop.

Python Program

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
```

```
# -----
# Step 1: Generate RSA Private Key
# -----
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

# Step 2: Obtain Public Key
public_key = private_key.public_key()

# -----
```

```

# Step 3: Original Message
# -----
message = b"Cryptography provides secure communication."

# -----
# Step 4: Generate Digital Signature
# -----
signature = private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("Original Message:")
print(message.decode())

print("\nDigital Signature:")
print(signature.hex())

# -----
# Step 5: Verify Original Message
# -----
try:
    public_key.verify(
        signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    print("\nOriginal Message Verification: SUCCESS")
    print("The message is authentic and has not been modified.")

except Exception:
    print("\nOriginal Message Verification: FAILED")

# -----
# Step 6: Modify Message

```

```

# -----
modified_message = b"Cryptography provides insecure communication."

# -----
# Step 7: Verify Modified Message
# -----
try:
    public_key.verify(
        signature,
        modified_message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    print("\nModified Message Verification: SUCCESS")

except Exception:
    print("\nModified Message Verification: FAILED")
    print("The message has been modified or the signature is invalid.")

```

Installation

If the **cryptography** library is not installed, execute:

```
pip install cryptography
```

Sample Output

Original Message:

Cryptography provides secure communication.

Digital Signature:

7f1c8b5a9d4e... [long hexadecimal signature]

Original Message Verification: SUCCESS

The message is authentic and has not been modified.

Modified Message Verification: FAILED

The message has been modified or the signature is invalid.

Note: The hexadecimal signature will be different each time because RSA-PSS uses randomized padding. This is normal and does not indicate an error.

Result

The RSA digital signature scheme was successfully implemented using Python. The original message was successfully verified using the public key. When the message was modified, signature verification failed.

Analysis

The experiment demonstrates that RSA digital signatures provide important security properties:

Security Property	Observation
Integrity	Modification of the message caused verification failure
Authentication	Public-key verification confirmed the valid signature
Non-repudiation	The signature is linked to the private key of the signer
Hashing	SHA-256 was used to process the message
Encryption Technique	RSA asymmetric cryptography was used

The use of **RSA-2048** provides strong security for the key-generation process, while **SHA-256** provides a secure message digest. RSA-PSS is preferred over older deterministic RSA signature padding schemes because it provides stronger security properties.

Conclusion

The RSA digital signature scheme was successfully implemented in Python. The experiment demonstrated that a valid signature can authenticate a message and detect modifications. Therefore, digital signatures are an important security mechanism for maintaining **integrity, authentication, and non-repudiation** in secure communication systems.

EXPERIMENT – 3

Title

Secure File-Integrity Verification System Using MD5 and SHA-256

Aim

To design and implement a secure file-integrity verification system using **MD5 and SHA-256**. The system generates hash values for a file, stores reference hashes, and later compares newly calculated hashes to determine whether the file has been modified.

Objectives

1. Read a file using Python.
2. Calculate its MD5 hash.
3. Calculate its SHA-256 hash.
4. Store the reference hash values.
5. Recalculate the hashes when verification is required.
6. Compare the reference and current hashes.
7. Detect whether the file has been modified.

Software Requirements

- Python 3.x
- Python `hashlib` library
- VS Code / IDLE / Jupyter Notebook

Theory

A **cryptographic hash function** converts input data of arbitrary size into a fixed-length hash value called a **digest**.

For file-integrity verification, a hash value is calculated for a file and stored as a reference value. When the file needs to be checked later, its hash is calculated again.

If the newly calculated hash is identical to the stored reference hash, the file is considered unchanged.

If the hash values are different, the file has been modified.

MD5

MD5 produces a **128-bit hash**, represented as a **32-character hexadecimal value**.

128 bits = 16 bytes = 32 hexadecimal characters

MD5 is fast but is considered cryptographically broken because practical collision attacks exist. Therefore, MD5 should not be used as the primary security mechanism for modern applications.

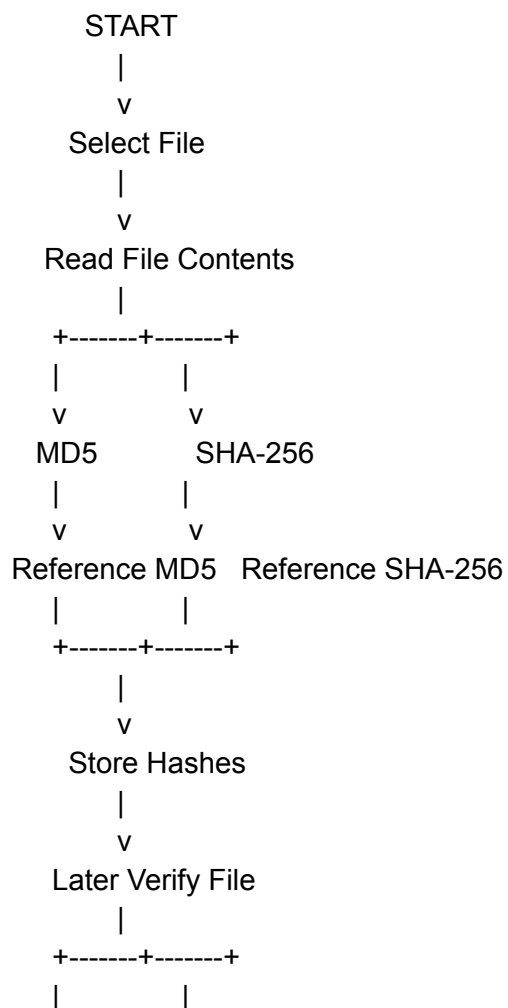
SHA-256

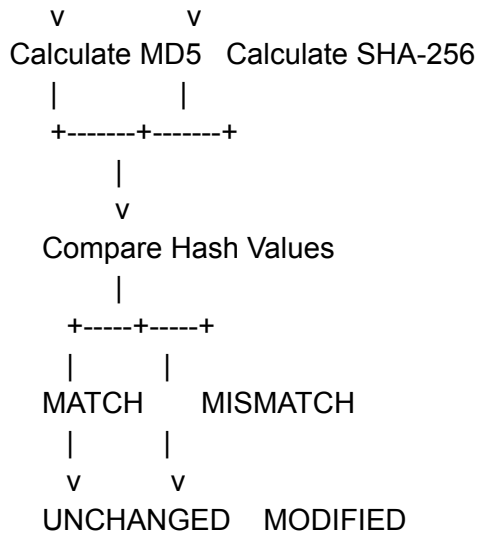
SHA-256 belongs to the SHA-2 family and produces a **256-bit hash**, represented as a **64-character hexadecimal value**.

256 bits = 32 bytes = 64 hexadecimal characters

SHA-256 provides significantly stronger collision resistance than MD5 and is recommended for modern file-integrity verification.

System Workflow





Algorithm

1. Start the program.
2. Select the file to be verified.
3. Open the file in binary mode.
4. Calculate its MD5 hash.
5. Calculate its SHA-256 hash.
6. Store both values as reference hashes.
7. Recalculate the hashes later.
8. Compare the current hashes with the reference hashes.
9. If both values match, report that the file is unchanged.
10. If either value differs, report that the file has been modified.
11. Stop.

Python Program

```

import hashlib
import json
import os

# -----
# Function to calculate file hashes
# -----

def calculate_hashes(filename):
    md5_hash = hashlib.md5()
    sha256_hash = hashlib.sha256()

    with open(filename, "rb") as file:

```

```

while True:
    data = file.read(4096)

    if not data:
        break

    md5_hash.update(data)
    sha256_hash.update(data)

return md5_hash.hexdigest(), sha256_hash.hexdigest()

# -----
# Function to create reference hashes
# -----
def create_reference(filename):
    md5_value, sha256_value = calculate_hashes(filename)

    reference = {
        "filename": filename,
        "md5": md5_value,
        "sha256": sha256_value
    }

    with open("reference_hashes.json", "w") as file:
        json.dump(reference, file, indent=4)

    print("\nReference hashes created successfully.")

    print("\nMD5:")
    print(md5_value)

    print("\nSHA-256:")
    print(sha256_value)

# -----
# Function to verify file integrity
# -----
def verify_file(filename):

    if not os.path.exists("reference_hashes.json"):
        print("Reference hash file not found.")
        return

```

```

with open("reference_hashes.json", "r") as file:
    reference = json.load(file)

current_md5, current_sha256 = calculate_hashes(filename)

print("\nReference MD5:")
print(reference["md5"])

print("\nCurrent MD5:")
print(current_md5)

print("\nReference SHA-256:")
print(reference["sha256"])

print("\nCurrent SHA-256:")
print(current_sha256)

if (current_md5 == reference["md5"] and
    current_sha256 == reference["sha256"]):

    print("\nFILE STATUS: UNCHANGED")
    print("File integrity verified successfully.")

else:
    print("\nFILE STATUS: MODIFIED")
    print("Warning: File integrity verification failed.")

# -----
# Main Program
# -----
print("=====")
print("  FILE INTEGRITY VERIFICATION SYSTEM")
print("=====")

filename = input("\nEnter file name: ")

if not os.path.exists(filename):
    print("File does not exist.")
else:

    print("\n1. Create Reference Hashes")
    print("2. Verify File Integrity")

```

```
choice = input("\nEnter your choice: ")

if choice == "1":
    create_reference(filename)

elif choice == "2":
    verify_file(filename)

else:
    print("Invalid choice.")
```

Sample Input

Create a text file named:

sample.txt

Example content:

This is a secure file integrity verification experiment.

Run the program:

Enter file name: sample.txt

1. Create Reference Hashes
2. Verify File Integrity

Enter your choice: 1

Sample Output – Reference Hash Generation

```
=====
FILE INTEGRITY VERIFICATION SYSTEM
=====
```

Enter file name: sample.txt

1. Create Reference Hashes

2. Verify File Integrity

Enter your choice: 1

Reference hashes created successfully.

MD5:

[32-character hexadecimal digest]

SHA-256:

[64-character hexadecimal digest]

The program creates:

reference_hashes.json

Example structure:

```
{
  "filename": "sample.txt",
  "md5": ".....",
  "sha256": "....."
}
```

File Verification – Unmodified File

After creating the reference hashes, run the program again:

Enter file name: sample.txt

1. Create Reference Hashes

2. Verify File Integrity

Enter your choice: 2

Sample result:

Reference MD5:

[original MD5 value]

Current MD5:

[original MD5 value]

Reference SHA-256:

[original SHA-256 value]

Current SHA-256:

[original SHA-256 value]

FILE STATUS: UNCHANGED

File integrity verified successfully.

File Verification – Modified File

Modify the contents of `sample.txt`, for example:

This is a MODIFIED file for integrity testing.

Run the verification again.

Sample output:

Reference MD5:

[original MD5 value]

Current MD5:

[different MD5 value]

Reference SHA-256:

[original SHA-256 value]

Current SHA-256:

[different SHA-256 value]

FILE STATUS: MODIFIED

Warning: File integrity verification failed.

Result

The file-integrity verification system was successfully implemented using Python's `hashlib` library. MD5 and SHA-256 hash values were generated and stored as reference values. When

the file was modified, the newly calculated hashes differed from the stored hashes, allowing the system to detect the modification.

Analysis

Feature	MD5	SHA-256
Output size	128 bits	256 bits
Hexadecimal length	32 characters	64 characters
Speed	Faster	Slightly slower
Collision resistance	Weak	Strong
Modern security	Not recommended for cryptographic security	Recommended
Python support	<code>hashlib.md5()</code>	<code>hashlib.sha256()</code>
Use in experiment	Comparison/reference	Preferred integrity mechanism

The experiment demonstrates that even a very small modification to a file changes its cryptographic hash. SHA-256 provides substantially stronger collision resistance than MD5 and is therefore more appropriate for modern security-sensitive file-integrity applications.

MD5 can still be useful for non-adversarial checksums and compatibility with legacy systems, but it should not be relied upon for security-critical integrity guarantees.

Conclusion

A secure file-integrity verification system was successfully implemented using MD5 and SHA-256. The system calculates and stores reference hash values and compares them with newly generated values during verification. Any modification to the file results in different hash values and is detected by the system.

The experiment demonstrates the practical application of cryptographic hashing for **data integrity verification**. Among the two algorithms, **SHA-256 is the preferred choice for modern security applications because of its stronger collision resistance**.

Overall Conclusion

The two experiments demonstrate important cryptographic security mechanisms.

The **RSA Digital Signature** experiment demonstrated how asymmetric cryptography can provide authentication, integrity, and non-repudiation. The **File Integrity Verification** experiment demonstrated how cryptographic hash functions can detect unauthorized modifications to files.

Together, these experiments show how cryptographic hashing and digital signatures are used in practical cybersecurity systems to protect information and establish trust between communicating parties.

References

1. William Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson.
2. Behrouz A. Forouzan, *Cryptography and Network Security*, McGraw-Hill.
3. Python Documentation – [hashlib](#) module.
4. Python Documentation – [cryptography](#) and RSA concepts.
5. National Institute of Standards and Technology (NIST), Secure Hash Standard (SHS).